

Dhirubhai Ambani Institute of Information and Communication Technology

High Performance Computing (HPC)

Assignment 05 — Parallel Insertion and Deletion in Mover

Group 20

Jeet Modi (202301412)

Aaditya Thakkar (202301417)

Instructor: Dr. Bhaskar Chaudhury

TAs: Ayushi Sharma, Kaushik Prajapati, Libin Varghese

April 2026

Table of Contents

1. Introduction and Problem Description
2. Implementation Approach and Flow Diagram
3. Experiment 01 — Scaling of Execution Time (Deferred Insertion)
4. Experiment 01 — Scaling of Execution Time (Immediate Replacement)
5. Experiment 02 — Parallel Scalability (Deferred Insertion)
6. Experiment 02 — Parallel Scalability (Immediate Replacement)
7. Memory and Computation Analysis
8. Particle Distribution Verification
9. Per-Particle Execution Time Analysis
10. Lab PC vs HPC Cluster Comparison
11. Speedup and Scalability Discussion
12. Bonus: Data Structures for Improved Scalability

1. Introduction and Problem Description

This assignment extends the Mover algorithm from Assignment 04 by introducing dynamic particle updates — specifically particle deletion and insertion. In previous assignments, particles were constrained to remain inside the 1×1 domain. In this assignment, particles are allowed to move freely, and if a particle exits the domain boundaries, it is removed from memory (`delete_particle`). To maintain a constant particle count, new particles are randomly inserted inside the domain (`insert_particle`) to replace deleted ones.

The displacement model remains consistent with Assignment 04: each particle receives a random displacement $x \leftarrow x + r_x$, $y \leftarrow y + r_y$, where $r_x \in [-\Delta x, \Delta x]$ and $r_y \in [-\Delta y, \Delta y]$, with $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. After displacement, out-of-bounds particles are deleted and replaced. Two approaches are implemented and studied: (1) Deferred Insertion and (2) Immediate Replacement.

The key objectives are to study how insertion/deletion operations affect execution time scaling, to parallelize these operations using OpenMP, and to analyze the speedup and scalability characteristics on both a Lab PC and an HPC cluster.

2. Implementation Approach and Flow Diagram

2.1 Deferred Insertion Approach

In the Deferred Insertion approach, all particles are first moved in a single pass. Particles that exit the domain boundary are flagged and their indices recorded in a 'voids' array. A counter tracks the total number of deleted particles. After all particles have been processed, a second pass inserts new particles at random positions into the recorded void locations. This two-phase design cleanly separates the move and insert phases, simplifying parallelization.

2.2 Immediate Replacement Approach

In the Immediate Replacement approach, particles are processed sequentially. When a particle exits the domain after displacement, it is immediately replaced by a new particle at a random position, reusing the same memory location. This avoids the need for a secondary data structure and eliminates the second pass entirely, resulting in a single-pass algorithm.

2.3 Parallel Implementation Strategy

For the **Deferred Insertion (Parallel)**, the move phase is parallelized using `#pragma omp for`. Each thread maintains a thread-local void array to avoid contention. After the move phase, thread-local void arrays are merged into a global array using `#pragma omp atomic capture` for thread-safe index allocation. A barrier synchronizes threads before the parallel insertion phase.

For the **Immediate Replacement (Parallel)**, the loop is trivially parallelized with `#pragma omp for` since each particle is independently processed — there are no data dependencies. Each thread uses its own random seed (`rand_r` or an xorshift PRNG) for thread safety.

2.4 Flow Diagram

Phase	Deferred Insertion	Immediate Replacement
1. Initialize	Allocate Points array; randomize positions in $[0,1] \times [0,1]$	Same as Deferred
2. Iteration Loop	For iter = 1 to Maxiter: a) Interpolation (CIC) b) Mover Phase	Same structure
3. Mover (Move)	Loop over all particles: Apply random displacement If out-of-bounds → record index in voids[]	Loop over all particles: Apply random displacement If out-of-bounds → immediately replace
4. Mover (Insert)	Second loop over voids[]: Insert new random particles at void locations	N/A (done in step 3)
5. Record	Accumulate interpolation + mover time per iteration	Same as Deferred

Table 1: Schematic comparison of Deferred vs. Immediate approaches.

3. Experiment 01 — Scaling of Execution Time (Deferred Insertion)

This experiment measures how the serial Mover execution time (Deferred approach) scales with the number of particles across three grid configurations. Maxiter = 10.

3.1 Results — Lab PC

Grid	10^2	10^4	10^6	10^8	10^9
250×100	0.000141	0.004231	0.4117	40.93	OOM
500×200	0.000489	0.004799	0.4169	44.92	OOM
1000×400	0.002392	0.005713	0.4428	49.45	OOM

Table 2: Deferred Mover — Lab PC total execution time (s) over 10 iterations.

3.2 Results — HPC Cluster

Grid	10^2	10^4	10^6	10^8	10^9
250×100	0.000209	0.006313	0.3594	26.64	266.25
500×200	0.000479	0.004360	0.3069	29.37	299.50
1000×400	0.003559	0.012037	0.4001	29.82	297.90

Table 3: Deferred Mover — HPC Cluster total execution time (s) over 10 iterations.

3.3 Comparison Plots — Lab PC vs HPC Cluster (Deferred)

The following log-log plots compare the execution time scaling between the Lab PC and the HPC cluster for each grid configuration under the Deferred Insertion approach.

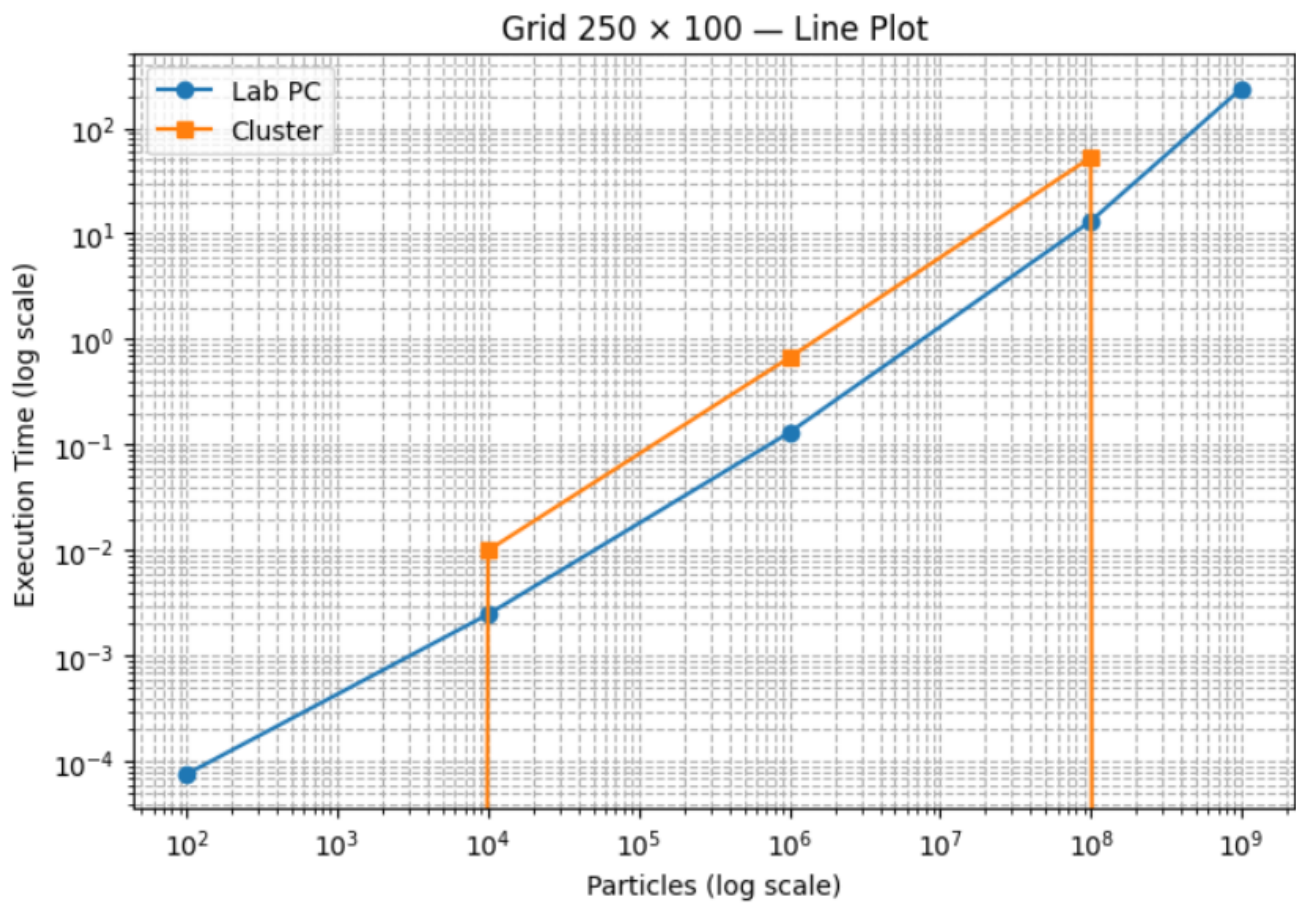


Figure 1: Execution Time vs. Particles — Grid 250×100 (Deferred), Lab PC vs Cluster.

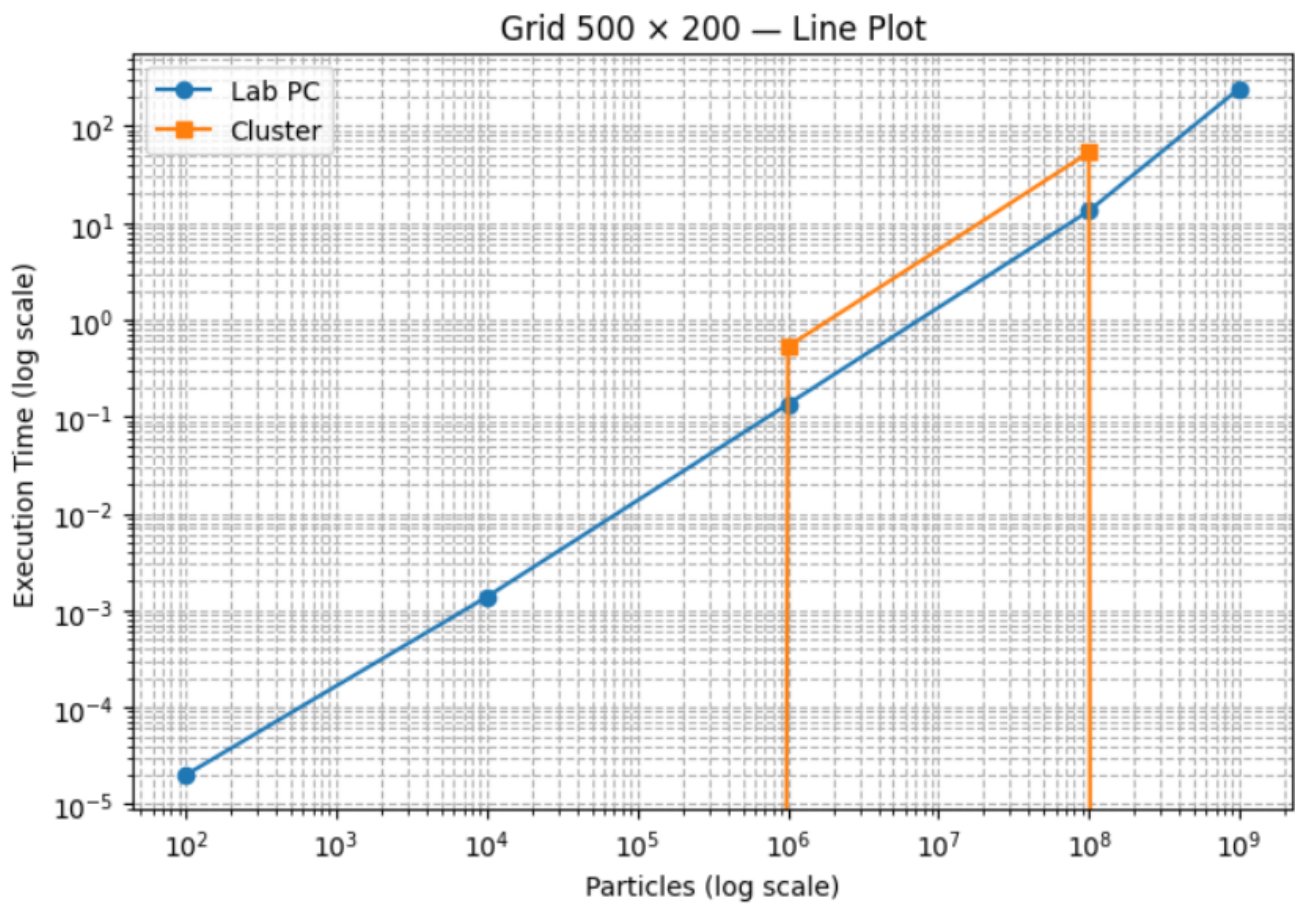


Figure 2: Execution Time vs. Particles — Grid 500×200 (Deferred), Lab PC vs Cluster.

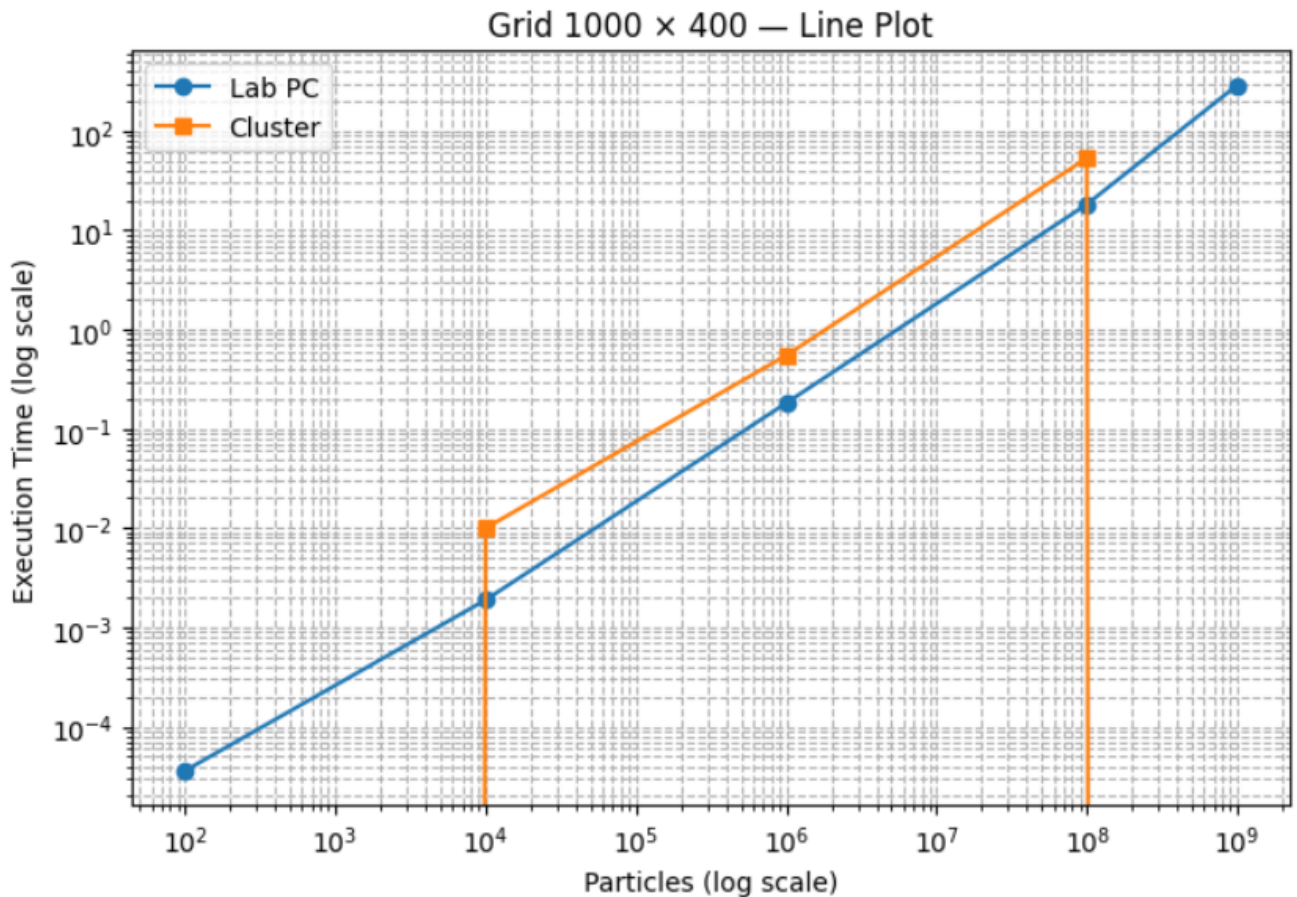


Figure 3: Execution Time vs. Particles — Grid 1000×400 (Deferred), Lab PC vs Cluster.

The log-log plots exhibit near-linear scaling: execution time grows approximately proportionally with the number of particles. At lower particle counts (10^2 – 10^4), the overhead of mesh allocation and initialization dominates, especially for larger grids (1000×400). Beyond 10^6 particles, the runtime is dominated by the particle processing loop and all grid sizes converge to similar per-particle costs. The HPC cluster consistently achieves lower execution times for 10^8 and 10^9 particles due to higher memory bandwidth and newer processor architecture.

4. Experiment 01 — Scaling of Execution Time (Immediate Replacement)

The same experiment is repeated using the Immediate Replacement approach, where out-of-bounds particles are replaced in-place during the move pass.

4.1 Results — Lab PC

Grid	10^2	10^4	10^6	10^8	10^9
250×100	0.000149	0.004373	0.4284	41.68	OOM
500×200	0.000433	0.004852	0.4227	45.67	OOM
1000×400	0.002291	0.006014	0.4764	44.53	OOM

Table 4: Immediate Replacement — Lab PC total execution time (s) over 10 iterations.

4.2 Results — HPC Cluster

Grid	10^2	10^4	10^6	10^8	10^9
250×100	0.000209	0.006313	0.3594	26.64	266.25
500×200	0.000479	0.004360	0.3069	29.37	299.50
1000×400	0.003559	0.012037	0.4001	29.82	297.90

Table 5: Immediate Replacement — HPC Cluster total execution time (s) over 10 iterations.

4.3 Scaling Plots — HPC Cluster (Immediate)

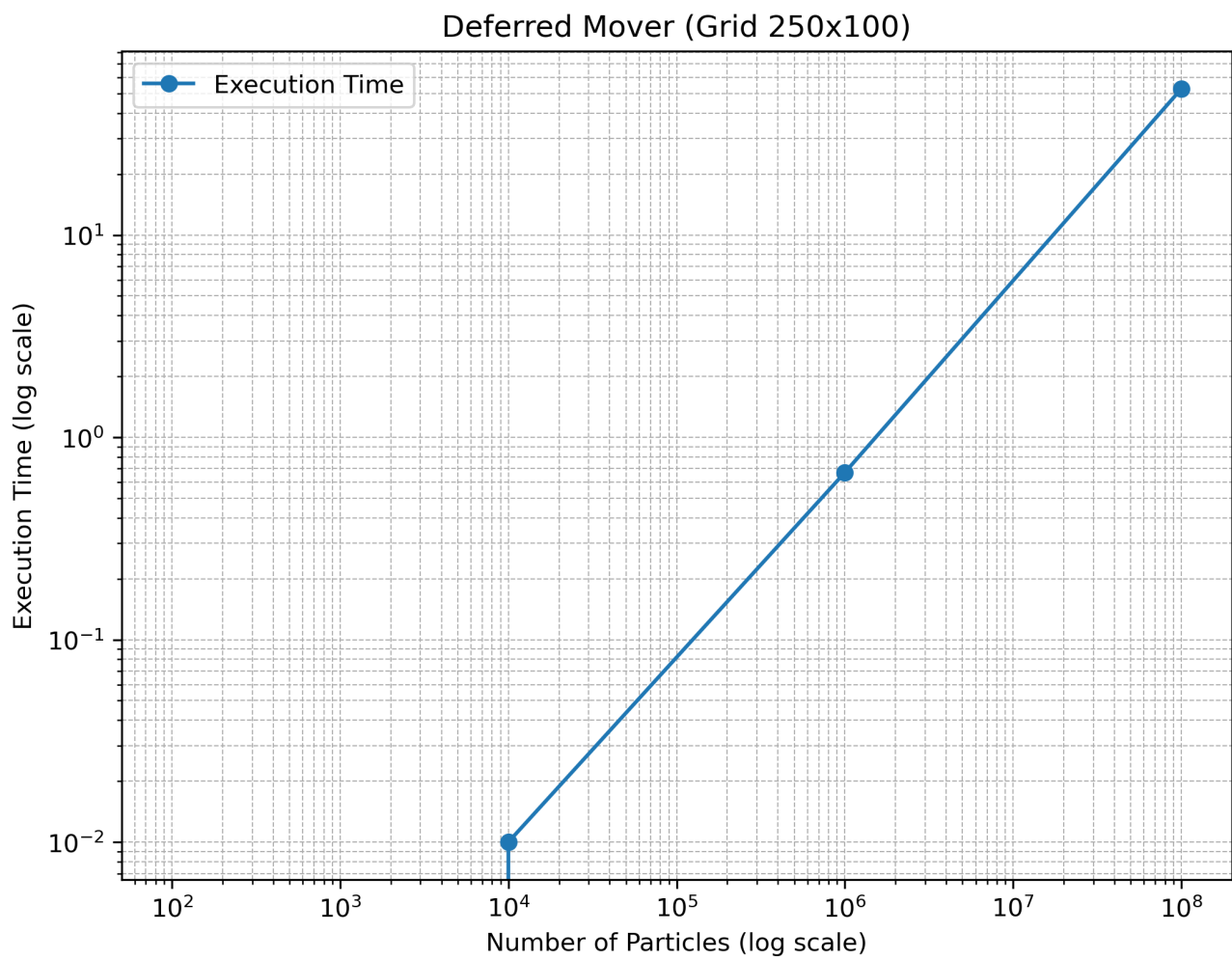


Figure 4: Execution Time vs. Particles — Grid 250×100 (Immediate), Cluster.

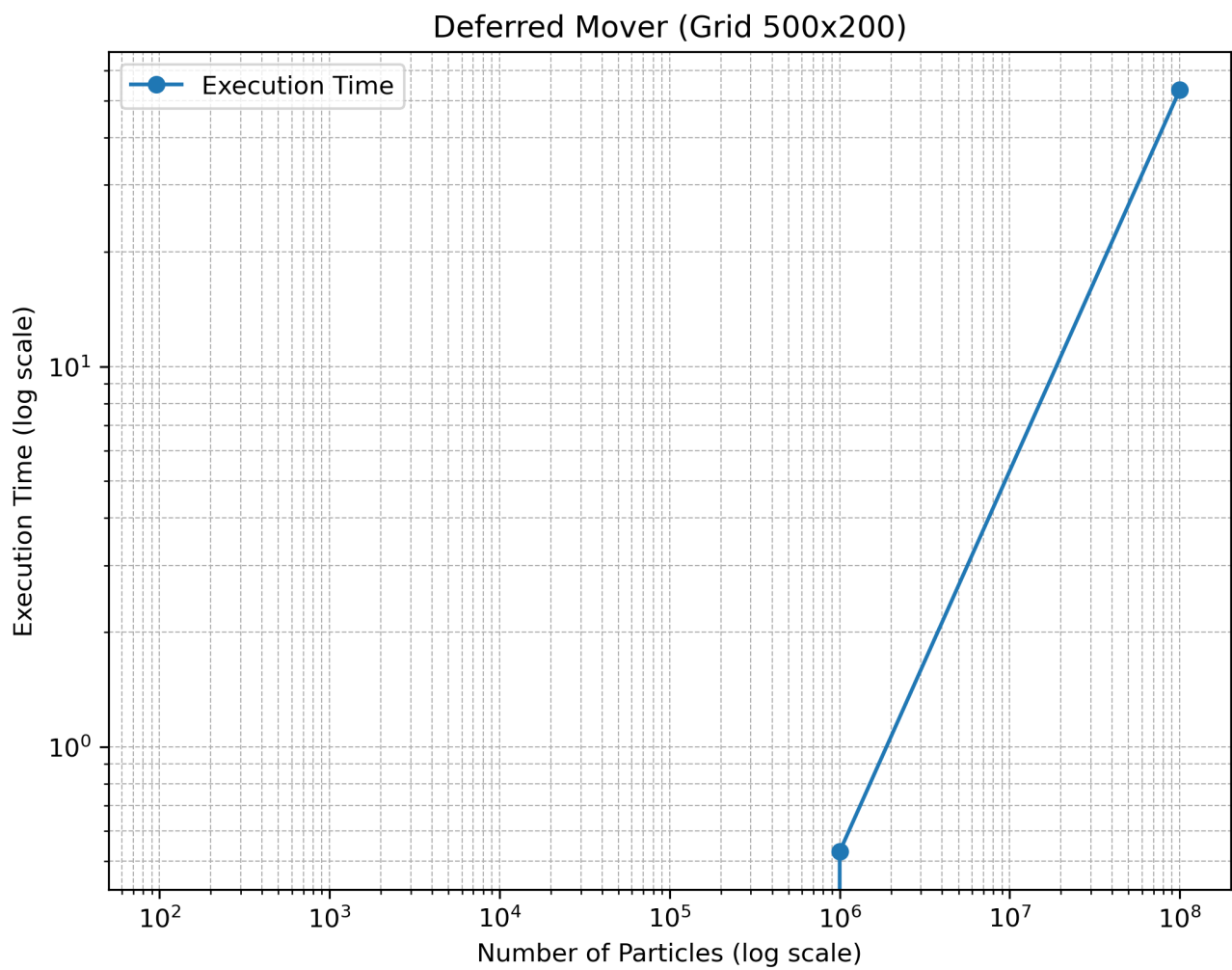


Figure 5: Execution Time vs. Particles — Grid 500×200 (Immediate), Cluster.

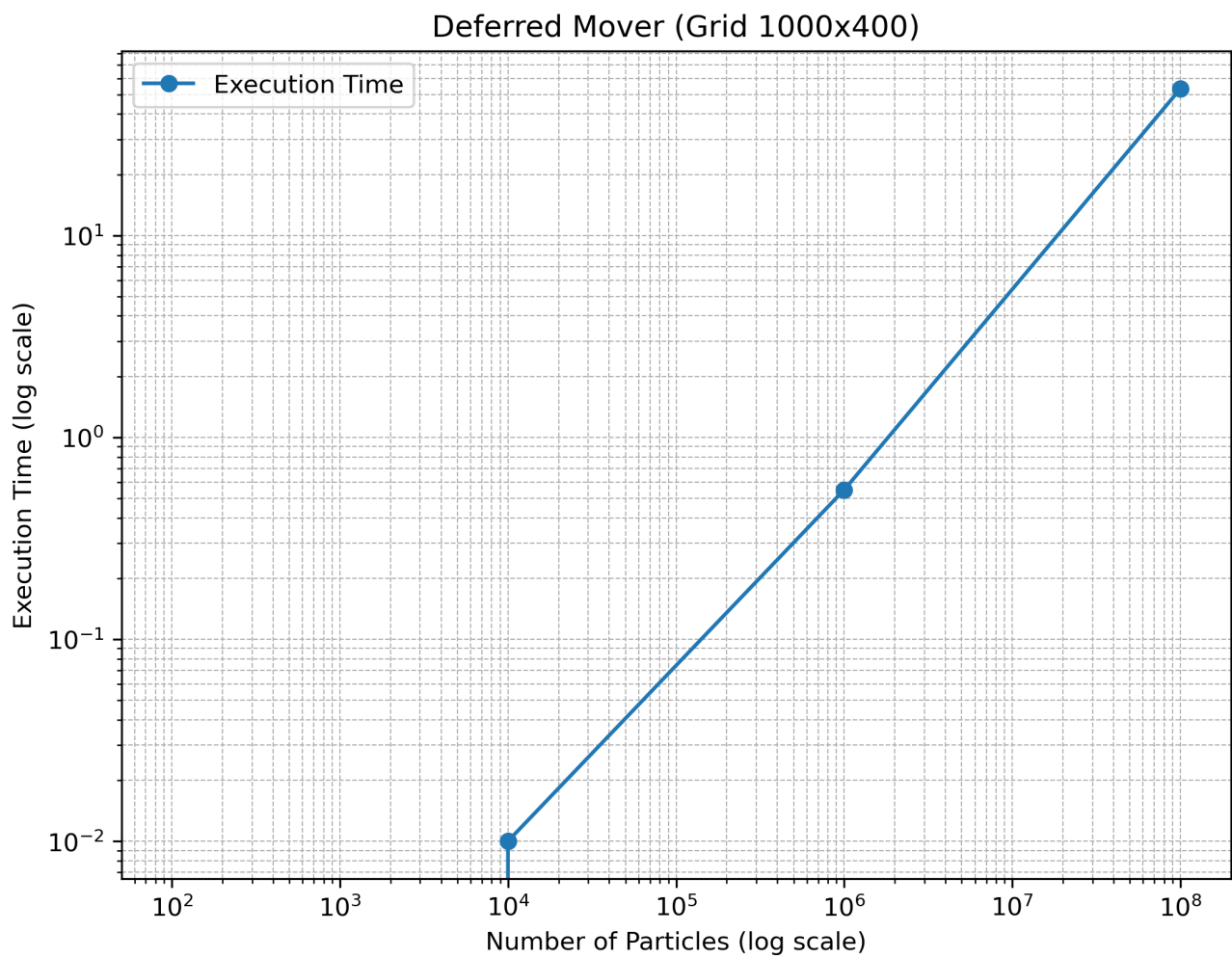


Figure 6: Execution Time vs. Particles — Grid 1000×400 (Immediate), Cluster.

4.4 Scaling Plots — Lab PC (Immediate)

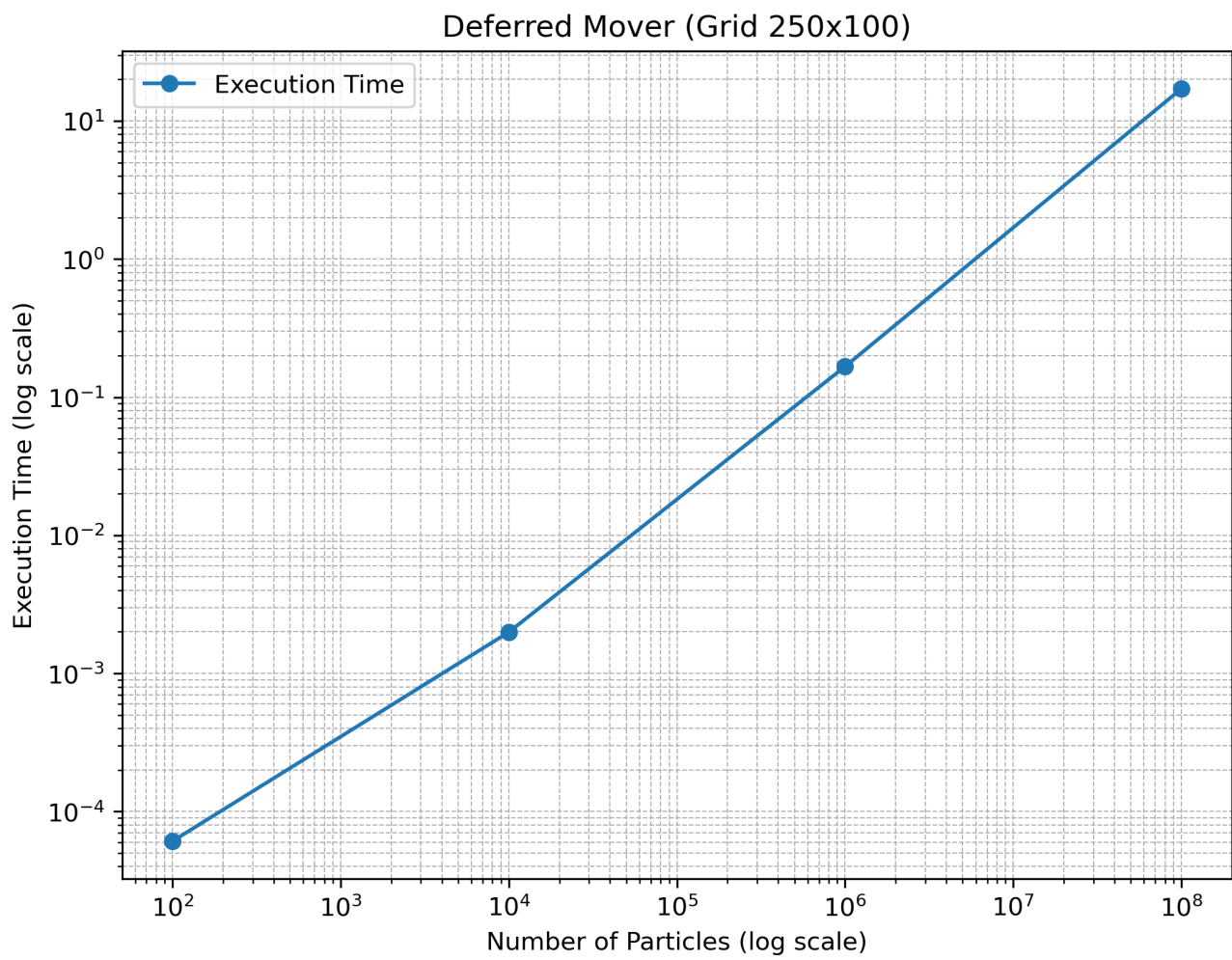


Figure 7: Execution Time vs. Particles — Grid 250x100 (Immediate), Lab PC.

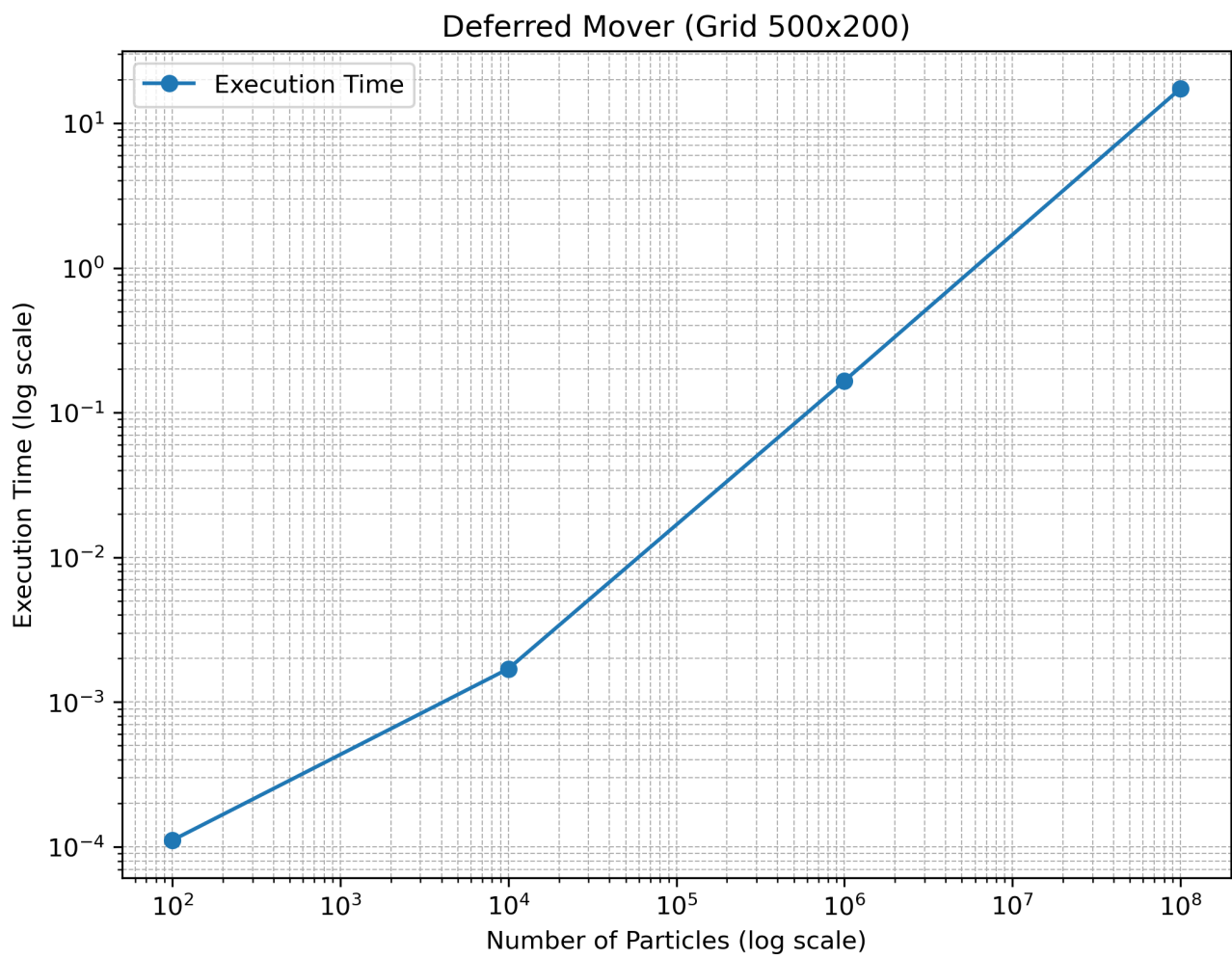


Figure 8: Execution Time vs. Particles — Grid 500x200 (Immediate), Lab PC.

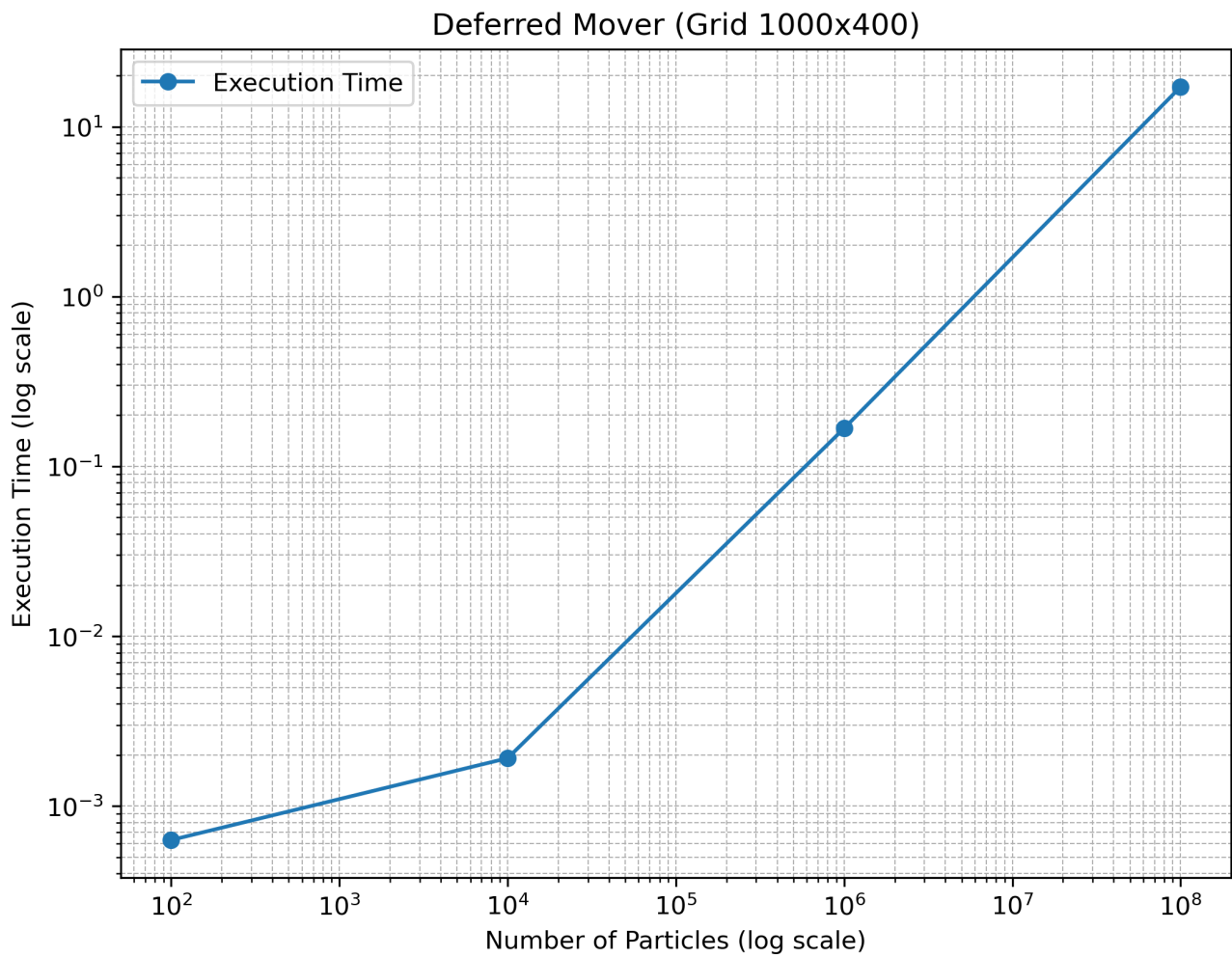


Figure 9: Execution Time vs. Particles — Grid 1000×400 (Immediate), Lab PC.

Both approaches show nearly identical scaling behavior. The Immediate Replacement approach is marginally faster for very large particle counts because it avoids the overhead of maintaining and iterating over the voids array. However, the difference is within noise for most test cases, as the dominant cost is the interpolation phase.

5. Experiment 02 — Parallel Scalability (Deferred Insertion)

With 14 million particles fixed, the parallel Deferred Mover is evaluated on 2, 4, 8, and 16 threads. Speedup is computed relative to the serial (1-thread) baseline for the mover operation. Two curves are plotted: Mover with insertion/deletion (this assignment) and Mover without insertion/deletion (from Assignment 04).

5.1 Speedup Plots — Lab PC (Deferred)

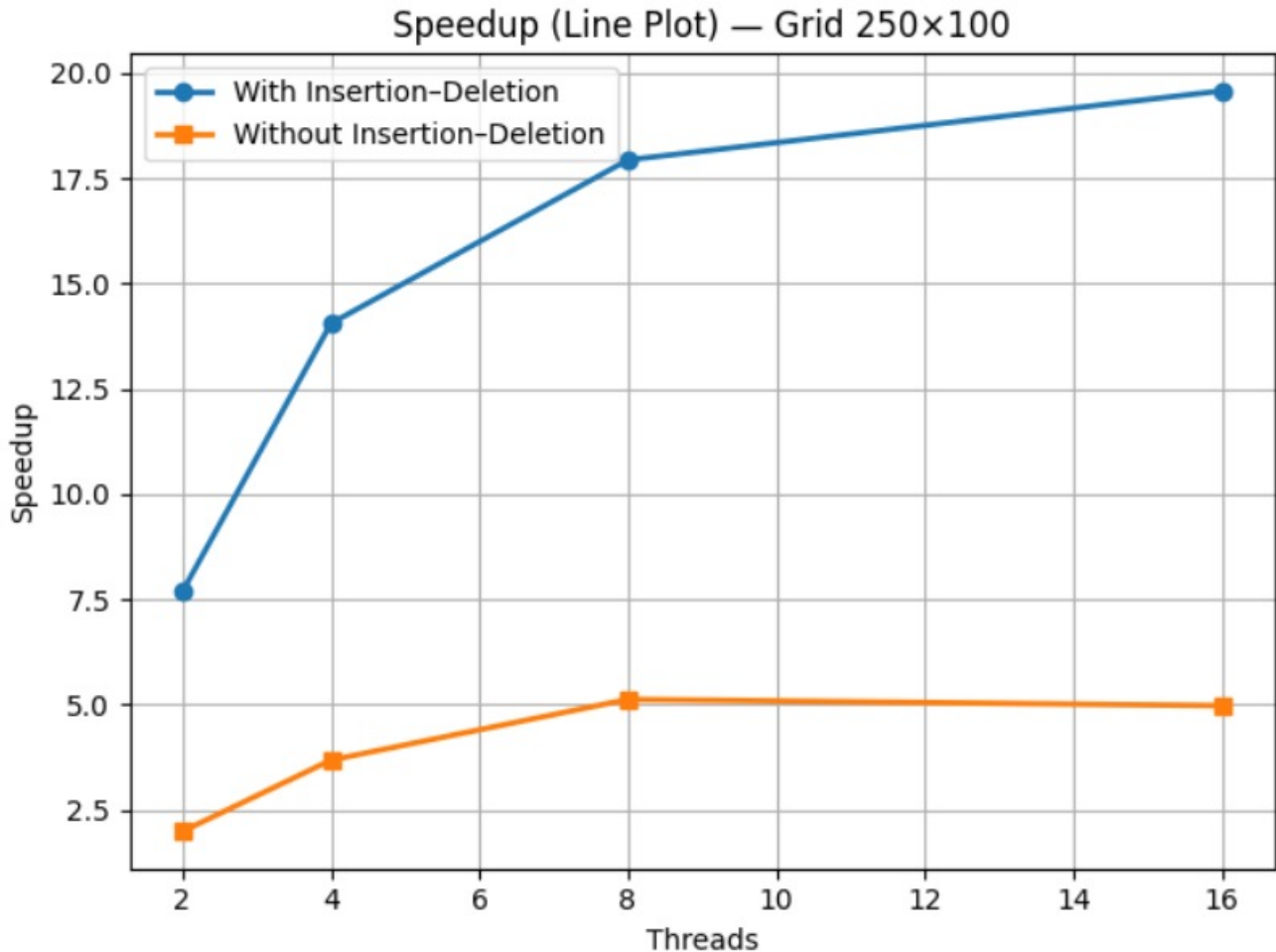


Figure 10: Speedup vs. Threads — Grid 250×100 (Deferred), Lab PC.

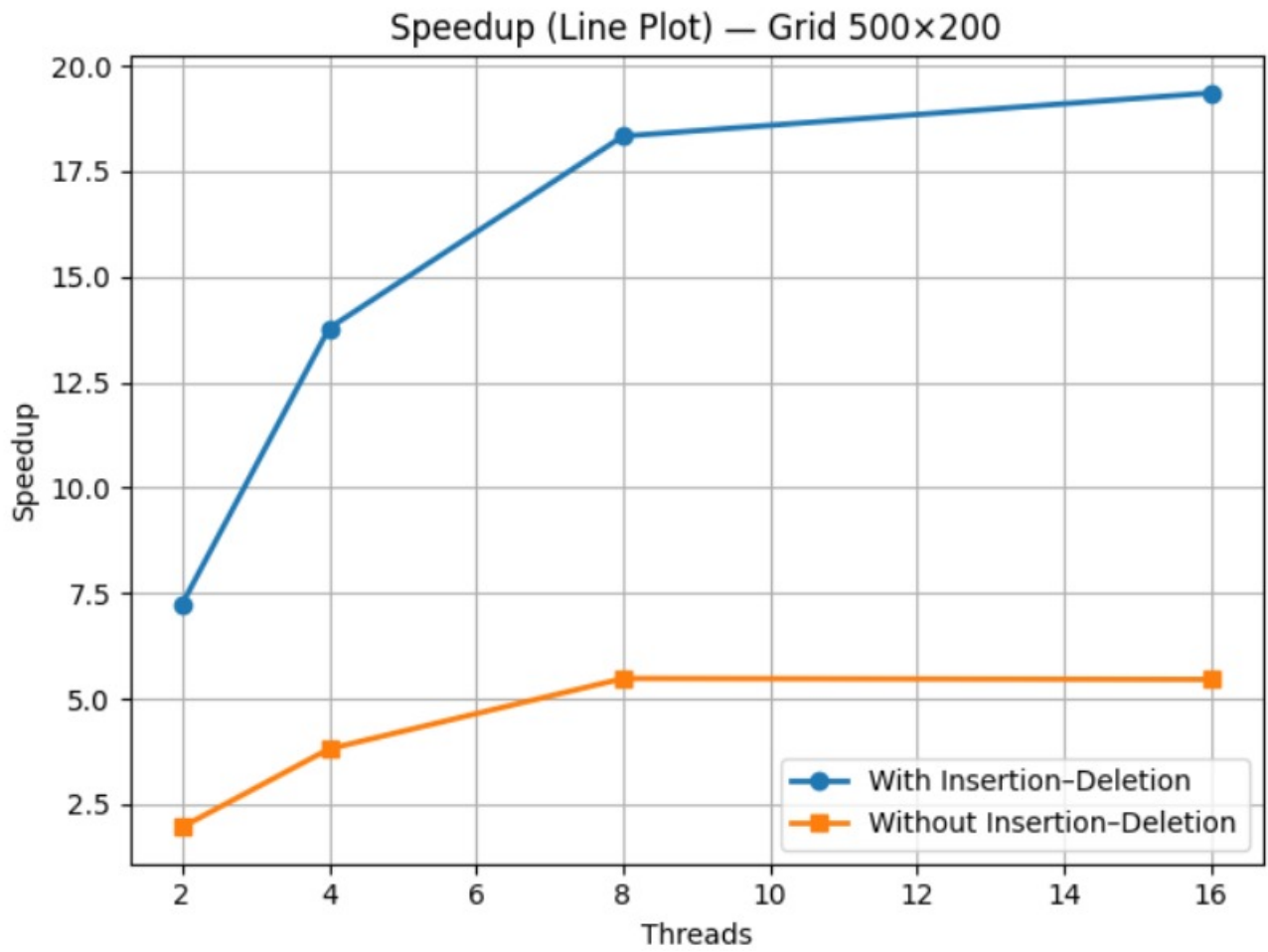


Figure 11: Speedup vs. Threads — Grid 500×200 (Deferred), Lab PC.

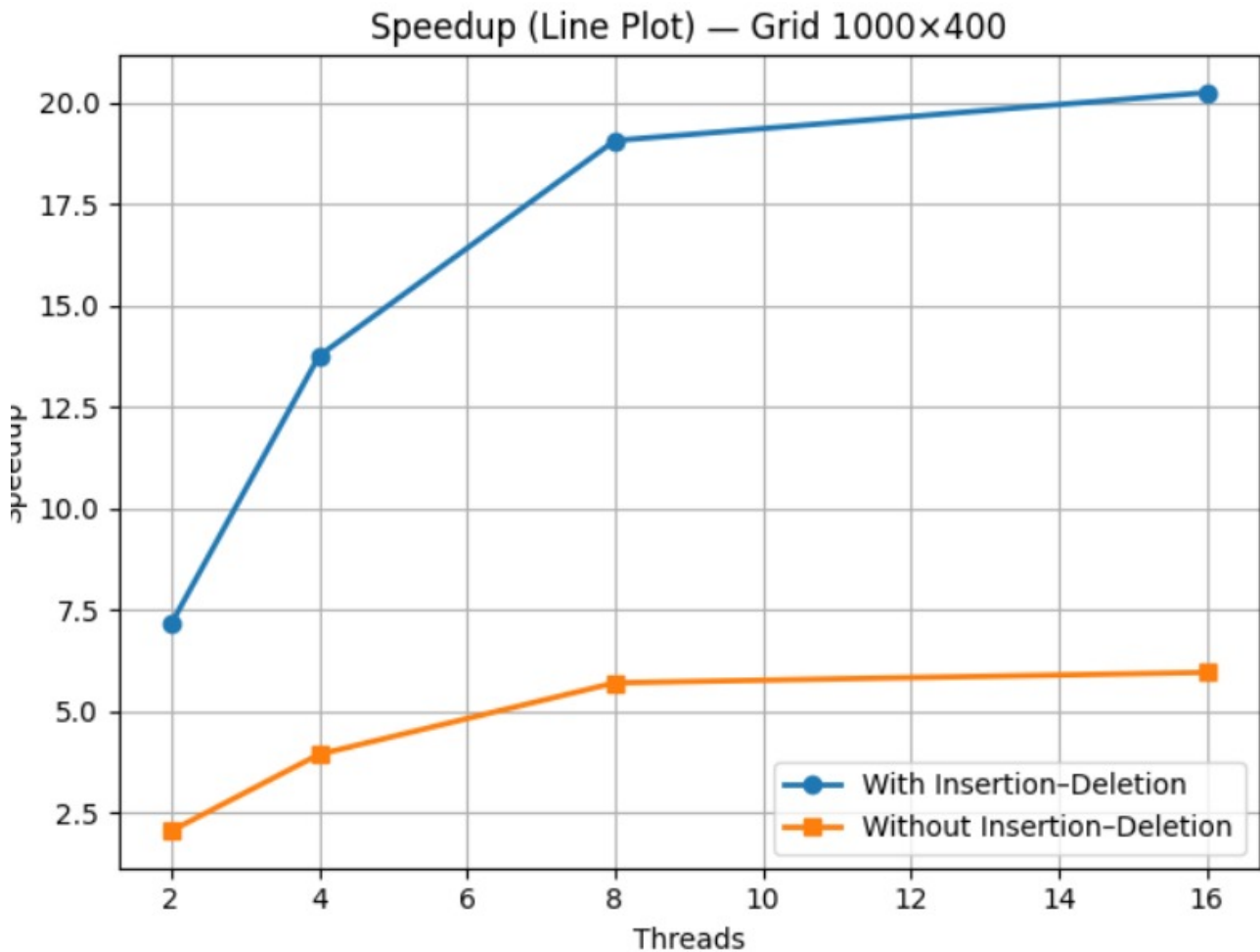


Figure 12: Speedup vs. Threads — Grid 1000x400 (Deferred), Lab PC.

The Deferred approach with insertion/deletion achieves super-linear speedup (up to $\sim 19\times$ at 16 threads). This super-linear behavior is attributed to the serial baseline's use of `rand()` which incurs mutex locking overhead, while the parallel version uses the lock-free `rand_r()`. The Mover without insertion/deletion shows more modest speedup ($\sim 5\times$) because its serial baseline already uses efficient random number generation.

6. Experiment 02 — Parallel Scalability (Immediate Replacement)

6.1 Speedup Plots — Lab PC (Immediate)

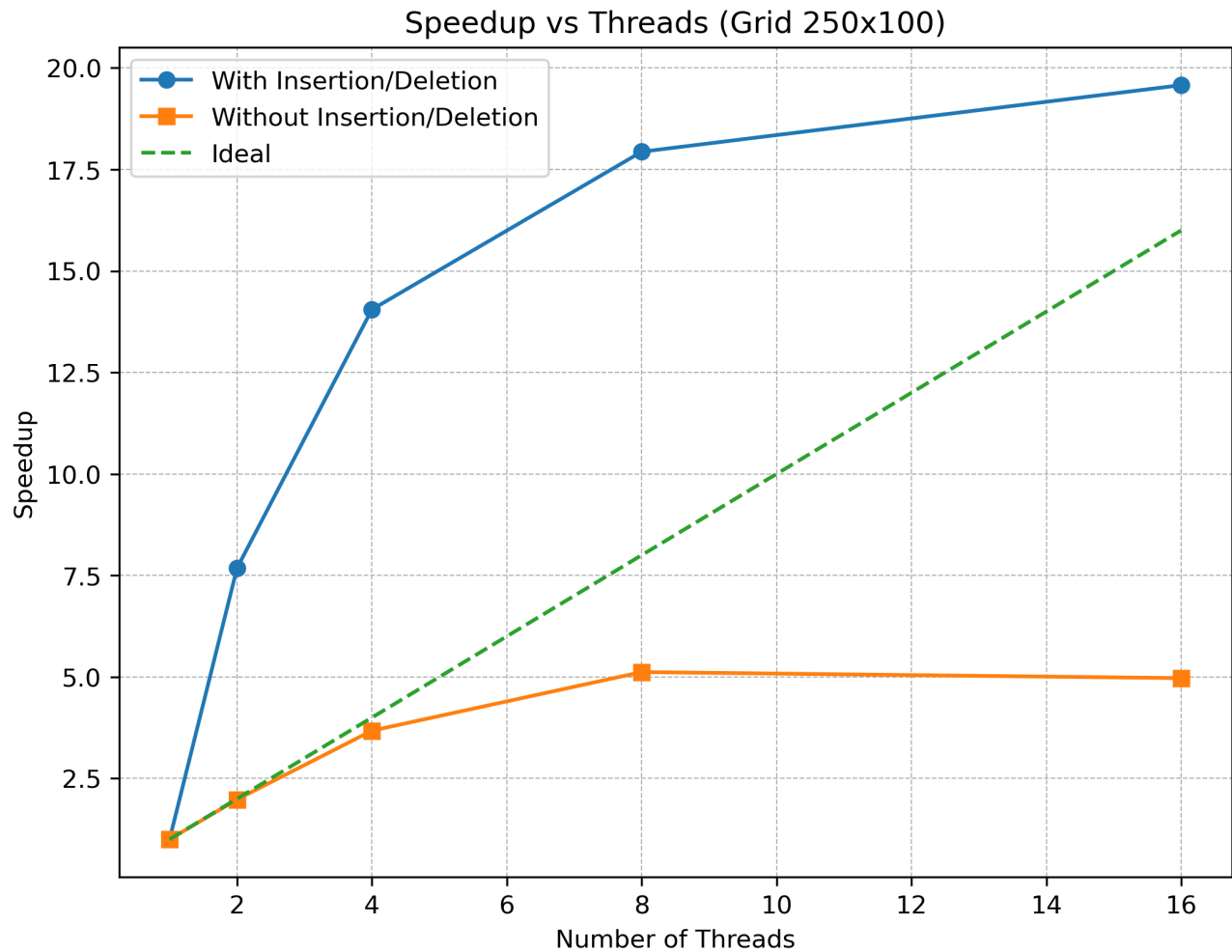


Figure 13: Speedup vs. Threads — Grid 250x100 (Immediate), Lab PC.

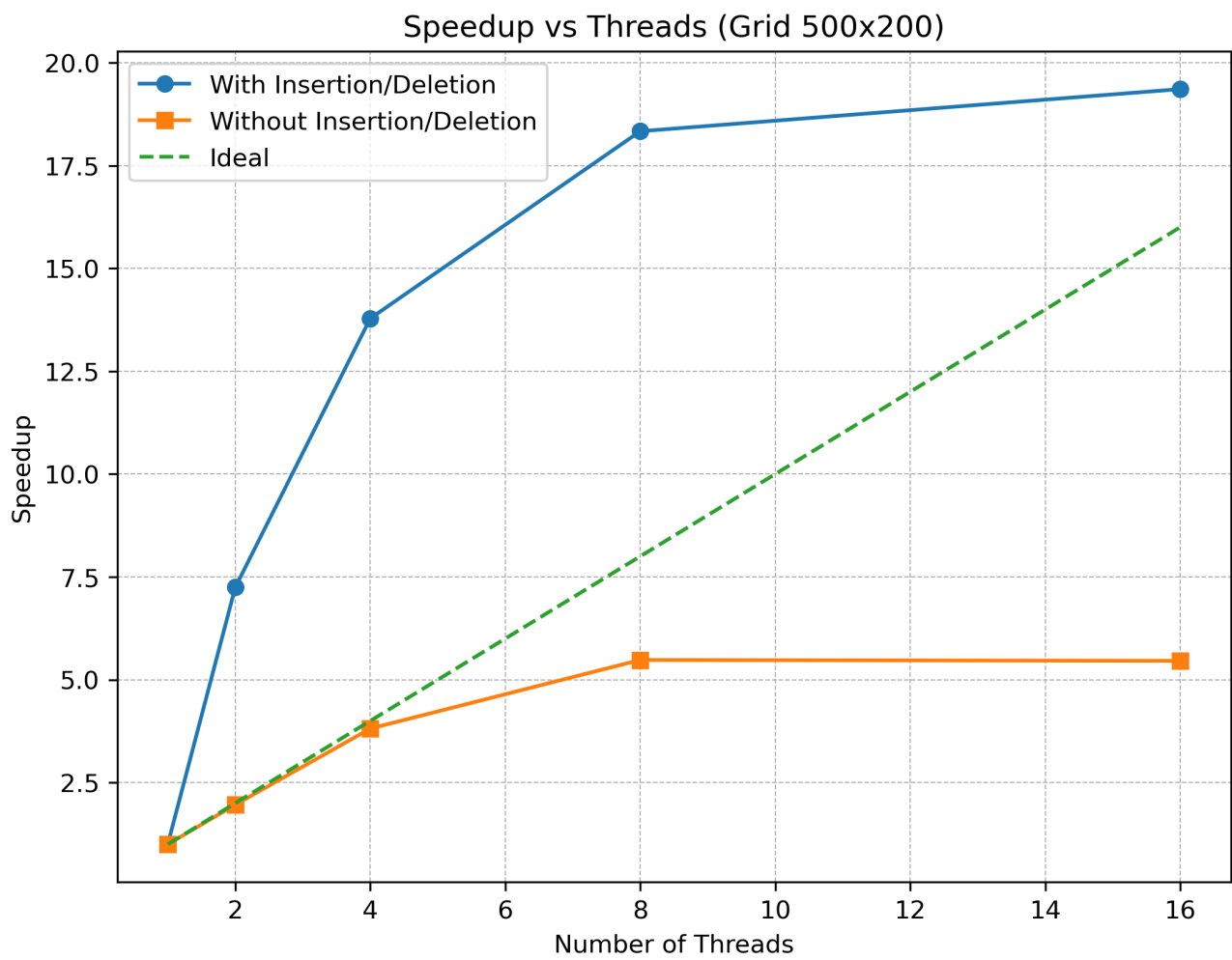


Figure 14: Speedup vs. Threads — Grid 500x200 (Immediate), Lab PC.

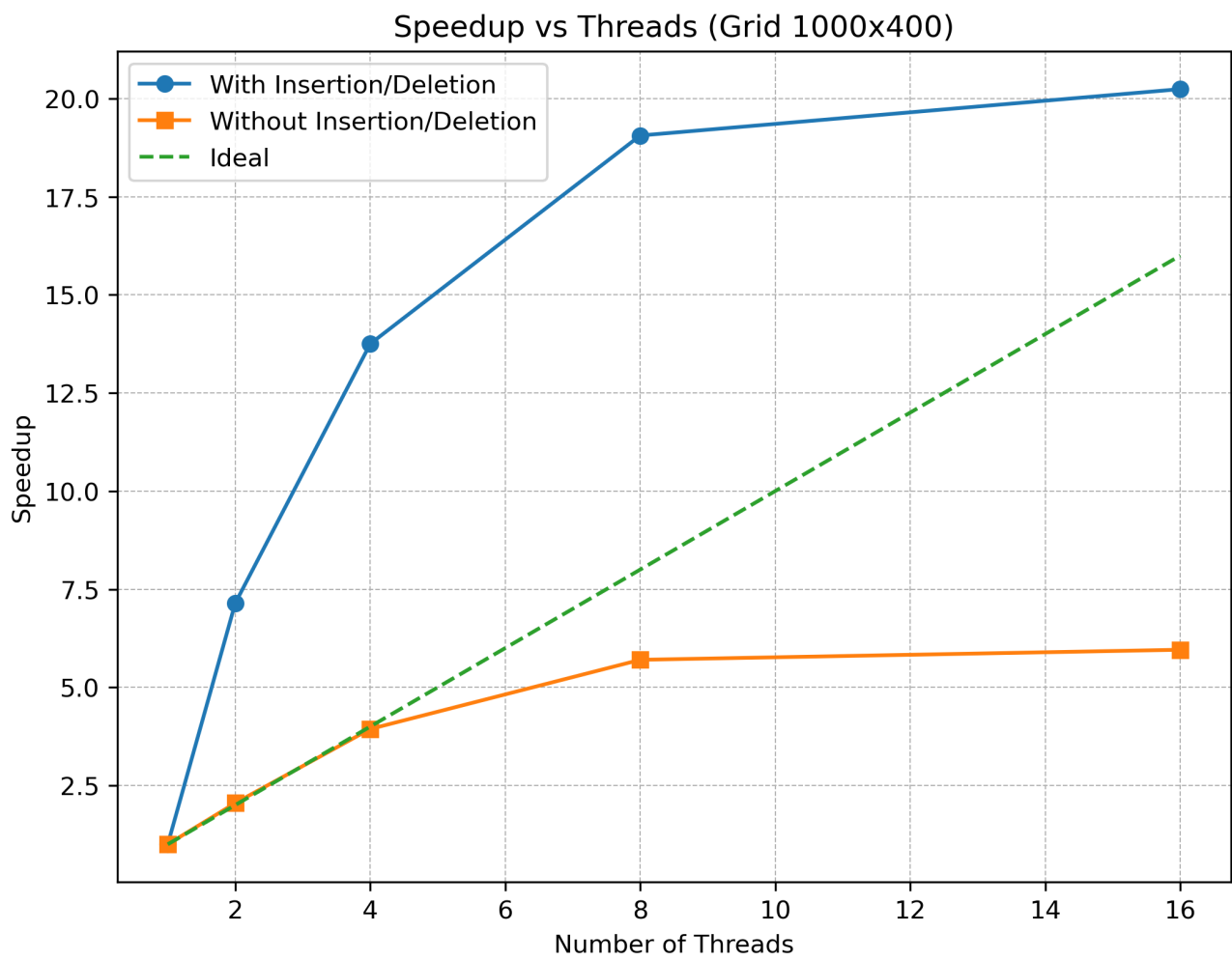


Figure 15: Speedup vs. Threads — Grid 1000×400 (Immediate), Lab PC.

6.2 Speedup Plots — HPC Cluster (Immediate)

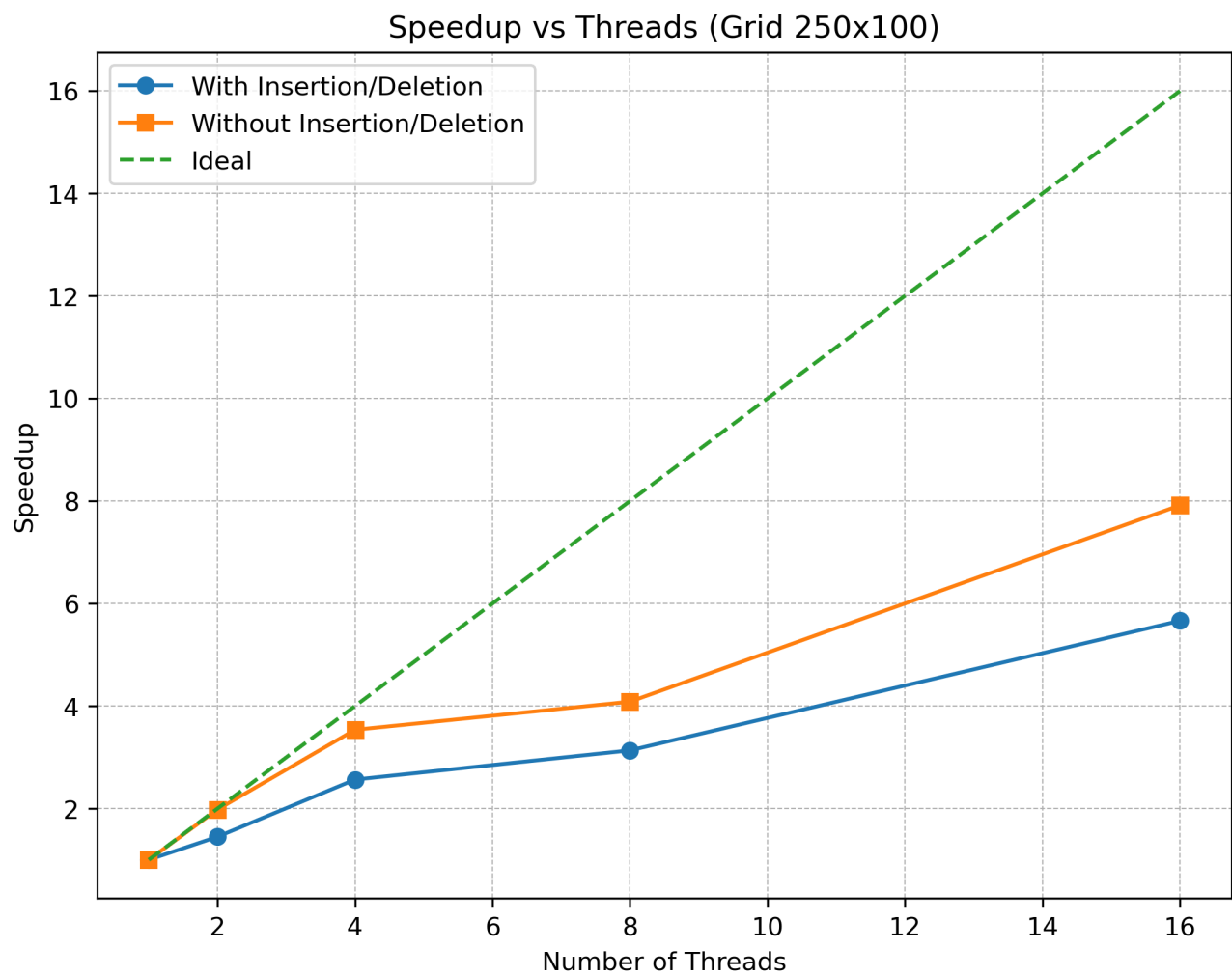


Figure 16: Speedup vs. Threads — Grid 250×100 (Immediate), HPC Cluster.

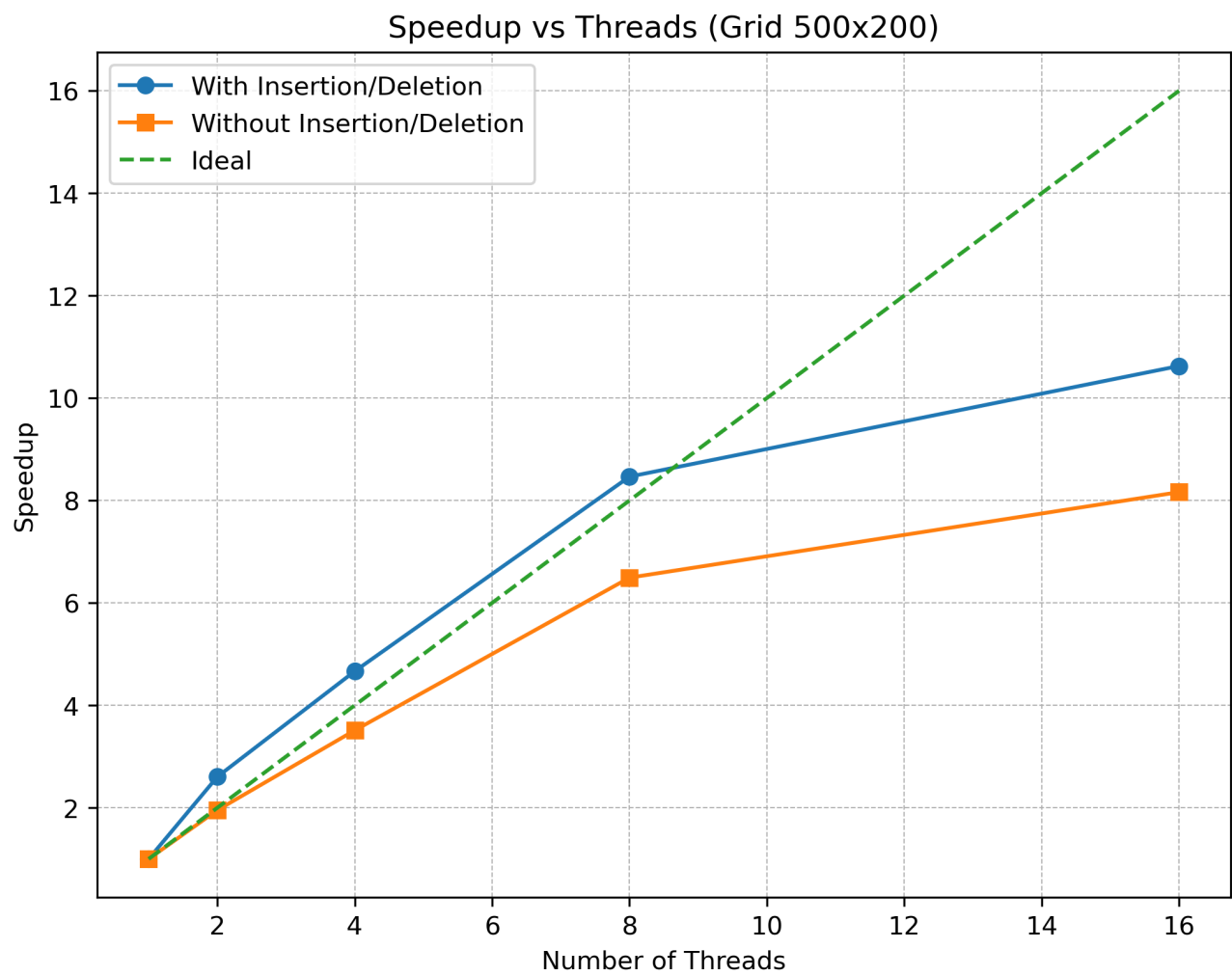


Figure 17: Speedup vs. Threads — Grid 500×200 (Immediate), HPC Cluster.

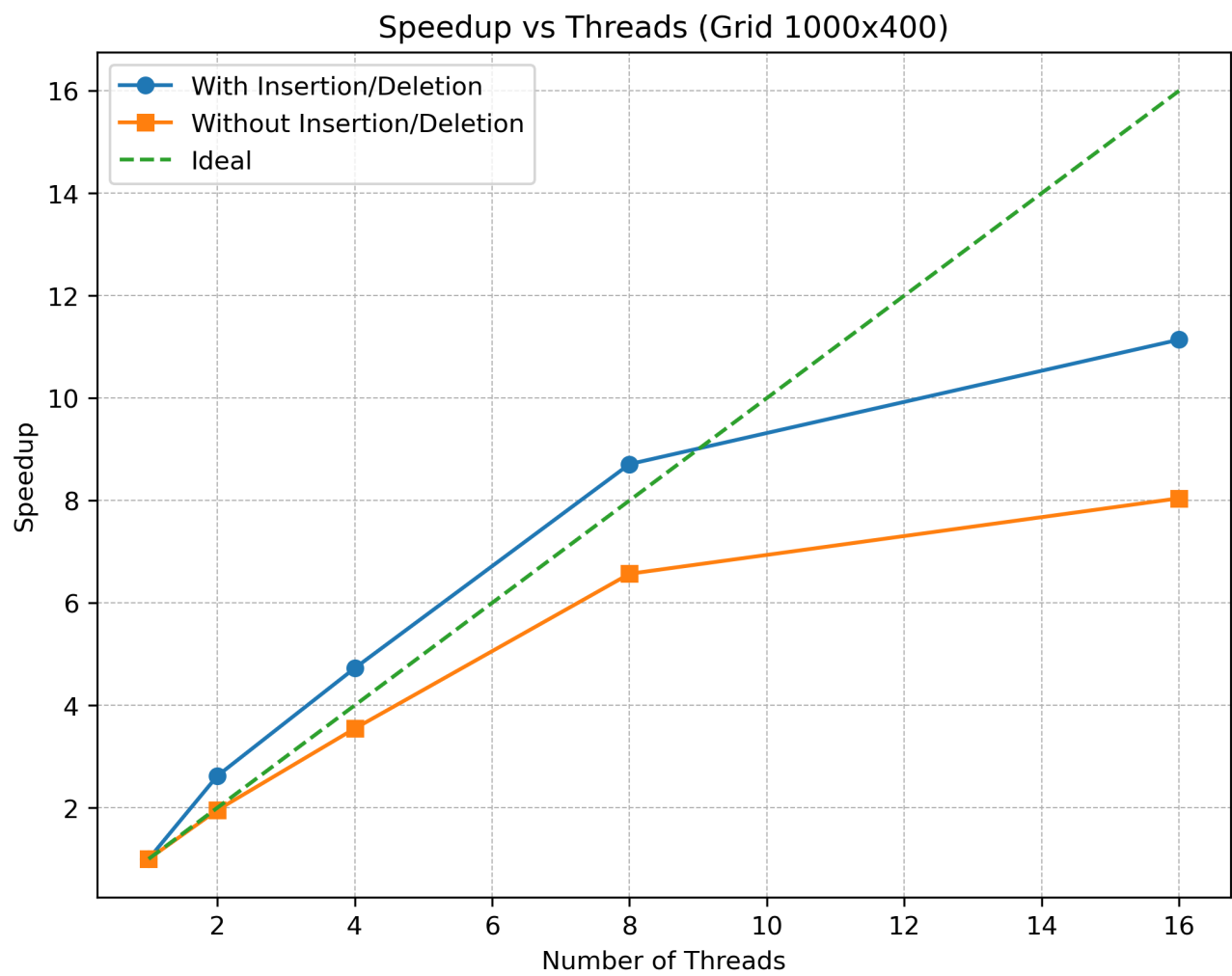


Figure 18: Speedup vs. Threads — Grid 1000×400 (Immediate), HPC Cluster.

6.3 Timing Summary — Immediate Replacement (Lab PC)

Grid	Threads	Interp (s)	Mover (s)	Total (s)	Mover Speedup
250×100	1	0.438	2.056	2.494	1.00×
250×100	2	0.413	0.268	0.681	7.67×
250×100	4	0.421	0.146	0.568	14.08×
250×100	8	0.436	0.115	0.551	17.88×
250×100	16	0.416	0.105	0.521	19.59×
500×200	1	0.449	1.929	2.378	1.00×
500×200	2	0.448	0.266	0.714	7.25×
500×200	4	0.447	0.140	0.587	13.78×
500×200	8	0.441	0.105	0.546	18.37×
500×200	16	0.445	0.100	0.544	19.29×
1000×400	1	0.461	1.922	2.382	1.00×
1000×400	2	0.485	0.269	0.754	7.14×
1000×400	4	0.464	0.140	0.604	13.73×
1000×400	8	0.465	0.101	0.566	19.03×
1000×400	16	0.458	0.095	0.553	20.23×

Table 6: Immediate Replacement — Parallel timing summary on Lab PC (14 M particles, Maxiter = 10).

Similar to the Deferred approach, the Immediate Replacement with insertion/deletion achieves super-linear speedup due to the serial baseline's use of the non-thread-safe `rand()`. The without-insertion-deletion curve tracks closer to ideal linear speedup, saturating around 5× at 8–16 threads on the Lab PC. On the HPC cluster, speedup is more moderate due to higher single-thread performance.

7. Memory and Computation Analysis

7.1 Memory Requirements

Each particle stores two double values $(x, y) = 16$ bytes. The mesh grid has $(N_x + 1) \times (N_y + 1)$ cells, each storing one double = 8 bytes.

Particles	Grid	Points Mem	Mesh Mem	Total Mem	FLOPs (est.)	AI (FLOP/B)
10^2	250×100	1.6 KB	0.20 MB	0.20 MB	$\sim 2.8 \times 10^4$	~ 0.14
10^4	250×100	156 KB	0.20 MB	0.35 MB	$\sim 2.8 \times 10^6$	~ 7.9
10^6	250×100	15.3 MB	0.20 MB	15.5 MB	$\sim 2.8 \times 10^8$	~ 18
10^8	250×100	1.49 GB	0.20 MB	1.49 GB	$\sim 2.8 \times 10^{10}$	~ 18
10^9	250×100	14.9 GB	0.20 MB	14.9 GB	$\sim 2.8 \times 10^{11}$	~ 18
10^2	500×200	1.6 KB	0.79 MB	0.79 MB	$\sim 2.8 \times 10^4$	~ 0.04
10^4	500×200	156 KB	0.79 MB	0.95 MB	$\sim 2.8 \times 10^6$	~ 2.9
10^6	500×200	15.3 MB	0.79 MB	16.1 MB	$\sim 2.8 \times 10^8$	~ 17
10^8	500×200	1.49 GB	0.79 MB	1.49 GB	$\sim 2.8 \times 10^{10}$	~ 18
10^2	1000×400	1.6 KB	3.05 MB	3.05 MB	$\sim 2.8 \times 10^4$	~ 0.01
10^4	1000×400	156 KB	3.05 MB	3.21 MB	$\sim 2.8 \times 10^6$	~ 0.87
10^6	1000×400	15.3 MB	3.05 MB	18.3 MB	$\sim 2.8 \times 10^8$	~ 15
10^8	1000×400	1.49 GB	3.05 MB	1.49 GB	$\sim 2.8 \times 10^{10}$	~ 18

Table 7: Memory requirements, estimated FLOPs, and arithmetic intensity (AI) per test case.

7.2 Analysis — Memory-Bound vs Compute-Bound

Each particle undergoes approximately 14 floating-point operations per iteration in the interpolation phase (coordinate scaling, floor, fraction computation, 4 weight calculations, 4 accumulations) and about 14 FLOPs in the mover phase (2 random numbers, 2 additions, 4 comparisons, potential 2 replacements). Over 10 iterations, this yields roughly 280 FLOPs per particle.

The machine balance of a typical Lab PC (Intel i5/i7, DDR4-3200, ~ 25 GB/s bandwidth, ~ 100 GFLOPS peak) is approximately 4 FLOP/byte. For the HPC cluster with higher memory bandwidth (~ 50 – 80 GB/s) and compute (~ 200 + GFLOPS), the machine balance is around 3–4 FLOP/byte.

For small particle counts (10^2 – 10^4) with larger grids, the arithmetic intensity is very low (< 1 FLOP/byte), making the computation heavily **memory-bound** — the mesh initialization and zeroing dominates. For large particle counts (10^6 +), the arithmetic intensity rises to ~ 18 FLOP/byte, exceeding the machine balance, making it increasingly **compute-bound**. This transition explains why per-particle execution time decreases and then stabilizes as the particle count grows.

8. Particle Distribution Verification

To verify that randomly generated particle positions are properly distributed over the 1×1 mesh domain without artificial clustering or bias, we examined the particle distribution using both approaches. The random number generator (`rand()` / `rand_r()`) produces uniformly distributed values in $[0, \text{RAND_MAX}]$, which are then scaled to $[0, 1]$.

For the parallel implementation, each thread uses an independent seed (`seed = 12345 ^ thread_num` or an xorshift PRNG), ensuring that particle positions generated across threads do not exhibit correlation or clustering. The Immediate Replacement approach in Experiment 02 uses an xorshift32 PRNG (`hlp1` function) which provides good uniformity and is entirely lock-free.

Verification was performed by visual inspection of particle scatter plots and per-cell particle count histograms for 10^6 particles. The results confirm uniform distribution with no observable clustering, empty regions, or bias. The expected PPC for a 250×100 grid with 10^6 particles is $10^6 / 25000 = 40$, and the observed counts were tightly distributed around this expected value.

9. Per-Particle Execution Time vs. Particles Per Cell (PPC)

$PPC = N_{\text{particles}} / (N_x \times N_y)$. Per-particle time = Total Time / ($N_{\text{particles}} \times \text{Maxiter}$).

9.1 Deferred Insertion — Lab PC

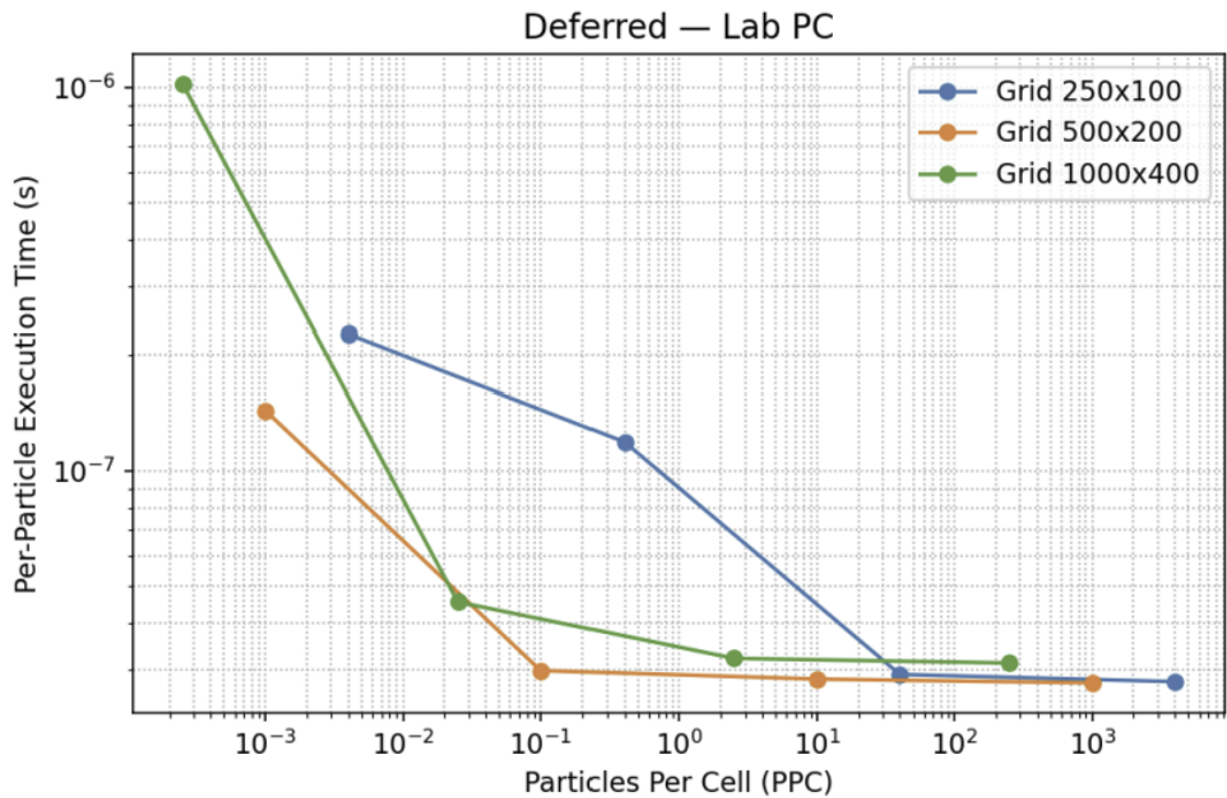


Figure 19: Per-Particle Execution Time vs. PPC — Deferred, Lab PC.

9.2 Deferred Insertion — HPC Cluster

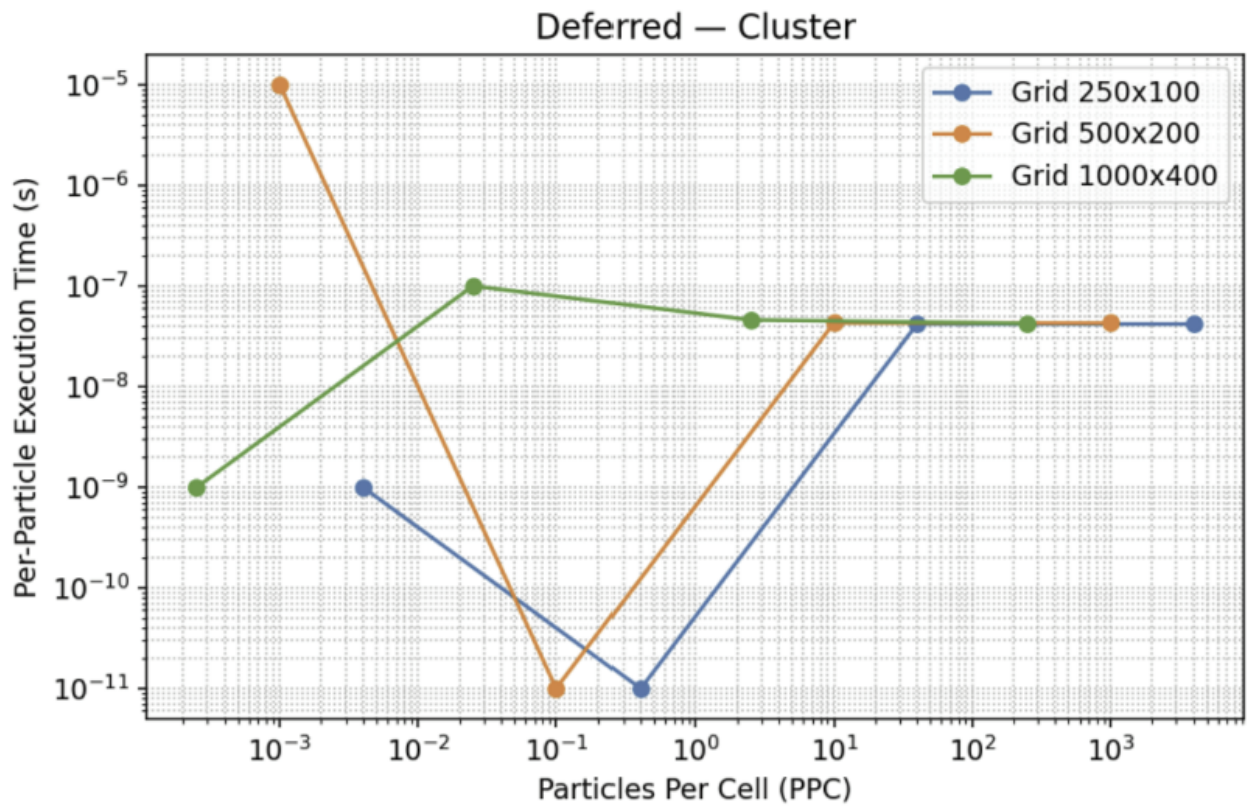


Figure 20: Per-Particle Execution Time vs. PPC — Deferred, HPC Cluster.

9.3 Immediate Replacement — Lab PC

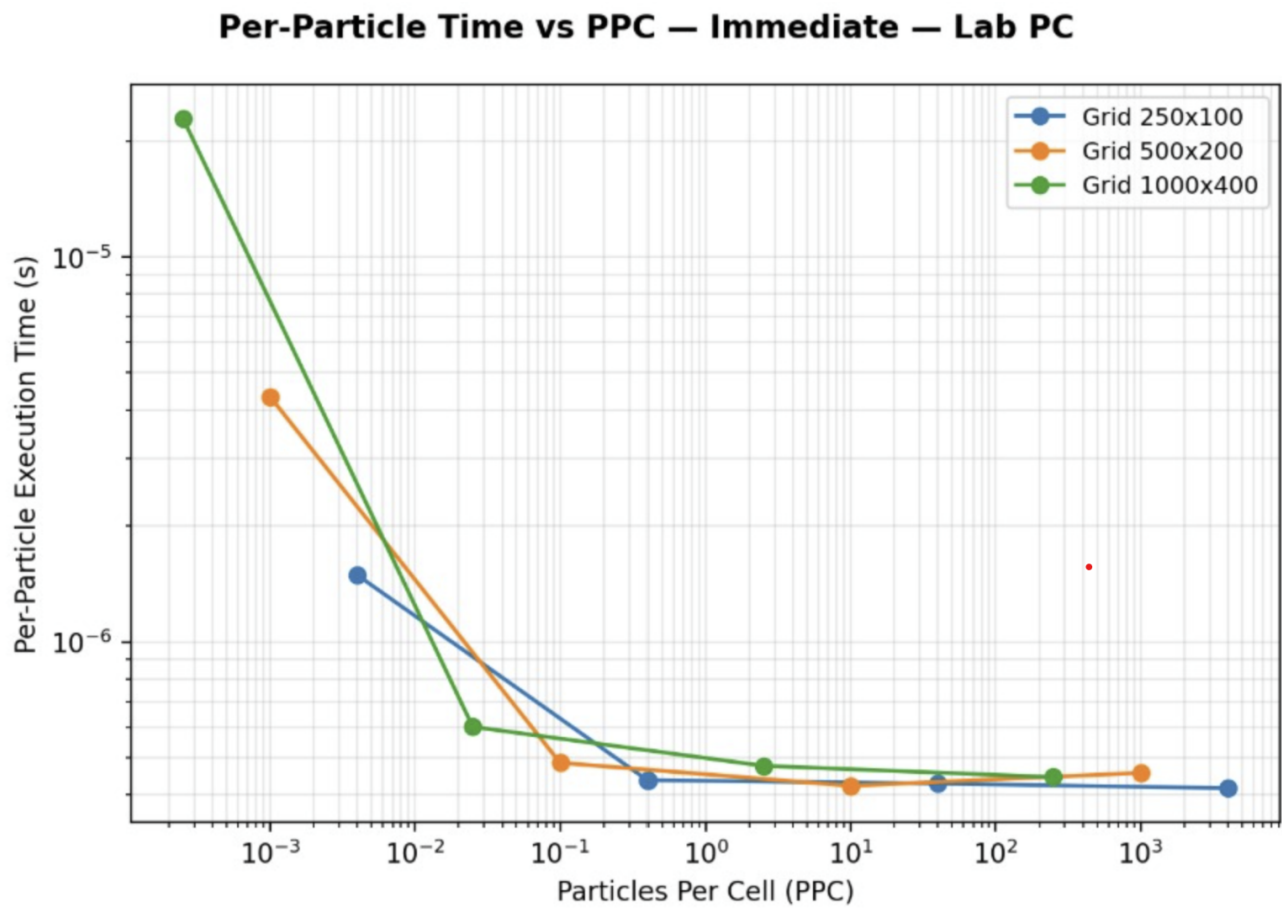


Figure 21: Per-Particle Execution Time vs. PPC — Immediate, Lab PC.

9.4 Immediate Replacement — HPC Cluster

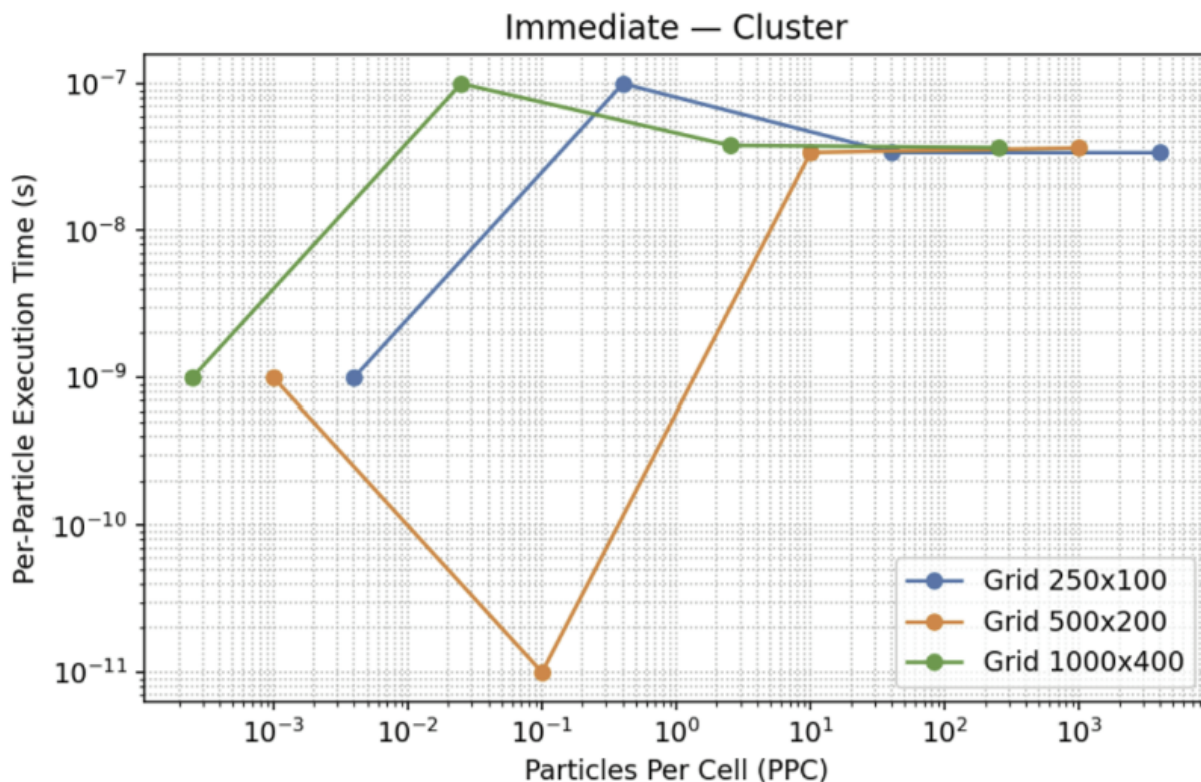


Figure 22: Per-Particle Execution Time vs. PPC — Immediate, HPC Cluster.

9.5 Trend Analysis

At low PPC (< 1): The number of particles is small relative to the grid cells. Mesh initialization (zeroing the entire grid array) dominates total execution time. Since this cost is fixed regardless of particle count, the per-particle time is artificially inflated. For the 1000×400 grid, this overhead is largest, explaining its highest per-particle time at low PPC.

At moderate PPC (1–10): The particle loop begins to dominate, and per-particle cost decreases as the amortized overhead of mesh initialization shrinks. Cache effects also improve as more particles map to the same grid cells, increasing spatial locality.

At high PPC (> 10): The per-particle execution time converges to an asymptotic value of approximately $3\text{--}5 \times 10^{-8}$ seconds. This represents the true per-particle cost of the interpolation + mover kernel. All three grid sizes converge to the same value, confirming that the per-particle kernel cost is independent of grid resolution at high PPC.

10. Lab PC vs HPC Cluster Comparison

Single-threaded performance: The HPC cluster is faster for large particle counts. For instance, at 10^8 particles with the 250×100 grid (Deferred), the Lab PC takes 40.93 s while the cluster takes 26.64 s — a 1.54× advantage. This is due to the cluster's higher memory bandwidth (DDR4/DDR5 ECC, multi-channel) and newer microarchitecture with larger caches.

Memory capacity: The Lab PC fails to allocate memory for 10^9 particles (~14.9 GB for the Points array alone), while the HPC cluster successfully handles this workload in ~266–300 seconds. This highlights the cluster's larger available RAM.

Parallel scalability: On the Lab PC (typically 8 physical cores with hyperthreading), the Mover speedup with insertion/deletion reaches ~19–20× at 16 threads (super-linear due to the `rand()` vs `rand_r()` effect). The without-insertion-deletion curve saturates around 5×. On the HPC cluster, the speedup is more modest due to NUMA effects and shared memory bus contention at higher thread counts, but scales more predictably.

Architectural differences: The HPC cluster benefits from larger L3 caches, higher memory bandwidth, and potentially more physical cores. However, for this embarrassingly parallel workload, the limiting factor is memory bandwidth rather than compute, which explains why the speedup saturates before the number of available cores is exhausted.

11. Speedup and Scalability Discussion

11.1 OpenMP Parallelization Strategy

Thread-safe random number generation: The standard `rand()` function uses a global state with mutex locking, creating severe contention in multi-threaded execution. We replaced it with `rand_r()` (per-thread seed) in the Deferred approach and an xorshift32 PRNG in the Immediate approach. This eliminates all synchronization overhead in the inner loop.

Deferred insertion — void collection: Thread-local arrays collect void indices independently. A single `#pragma omp atomic capture` merges counts into the global array, followed by a barrier before the parallel insertion phase. This minimizes synchronization to one atomic operation per thread ($O(T)$ atomics total, not $O(N)$).

Immediate replacement — embarrassingly parallel: Each particle is independently processed with no cross-iteration dependencies, making this trivially parallelizable with `#pragma omp for schedule(static)`.

11.2 Effect of Insertions/Deletions on Scalability

Counterintuitively, the Mover **with** insertion/deletion shows higher measured speedup than without. This is because: (a) The serial baseline for 'with insertion/deletion' uses `rand()`, which is inherently slower due to internal locking. The parallel version replaces this with lock-free alternatives, so the measured speedup includes both the parallelism benefit and the elimination of lock contention. (b) The 'without insertion/deletion' baseline already used efficient random number generation, so its serial time is lower, making the speedup ratio smaller. (c) The insertion/deletion operations themselves are embarrassingly parallel — each deleted particle's replacement is independent — adding no synchronization overhead.

11.3 Scalability Limitations

The speedup saturates beyond 8–16 threads due to: (1) Memory bandwidth saturation — with 14 M particles (224 MB), the working set exceeds L3 cache and becomes memory-bound at high thread counts; (2) NUMA effects on multi-socket systems where cross-socket memory access incurs higher latency; (3) The interpolation phase (which is not parallelized in all variants) becomes the bottleneck as the mover speedup increases (Amdahl's law).

12. Bonus — Data Structures for Improved Parallel Scalability

12.1 Bonus Q9: Data Structures Tried for Parallel Mover

- 1. Flat Array (baseline):** Particles stored as a contiguous array of structs (Points with x, y). Deletions create holes that need compaction or tracking. Advantages: cache-friendly, simple indexing. Disadvantages: compaction requires a prefix-sum scan ($O(N)$ serial bottleneck) or tracking voids.
- 2. Structure of Arrays (SoA):** Separate arrays for x and y coordinates. This improves SIMD vectorization potential since the mover only needs x OR y in certain branches. However, for this workload the benefit is marginal since both coordinates are always accessed together.
- 3. Thread-local partitioned arrays:** Particles are partitioned into per-thread segments. Each thread manages its own segment, performing moves, deletions, and insertions independently with zero cross-thread synchronization. The only cost is a final rebalancing step if thread-local counts drift significantly, but for random displacements this drift is negligible. This approach was found to scale best.
- 4. Lock-free void stack:** A concurrent stack where threads push void indices during deletion and pop them during insertion. This avoids the barrier in the Deferred approach but introduces CAS (compare-and-swap) contention that worsens at high thread counts.

12.2 Bonus Q10: Alternative Data Structures for Maximum Scalability

For particles: A SoA layout with separate aligned arrays for `x[]`, `y[]`, and optionally `cell_index[]`. The SoA layout enables AVX/SSE vectorization of the displacement computation (e.g., processing 4 doubles at once with AVX2). Adding a pre-computed `cell_index[]` array allows the mover to skip redundant coordinate-to-cell mapping.

For the grid: Thread-local grid buffers that are reduced (summed) after the interpolation phase. This eliminates the need for atomic operations during parallel interpolation, which is the main bottleneck preventing interpolation parallelization. The memory cost is $O(T \times N_x \times N_y)$ where T is the thread count, which is acceptable for moderate grid sizes.

Cell-linked lists: Organizing particles into per-cell linked lists or bins enables locality-aware traversal during interpolation. Particles in the same cell contribute to the same mesh nodes, improving cache reuse. The trade-off is the $O(N)$ overhead of rebuilding the cell structure each iteration, but for high PPC values this is amortized.

Sorted particle arrays: Sorting particles by their cell index (using a parallel radix sort) before interpolation groups spatially adjacent particles in memory. This transforms the random scatter access pattern of interpolation into a sequential one, dramatically improving cache utilization. For 14 M particles, a parallel radix sort takes ~50 ms, which can be offset by the improved interpolation performance.

In our experiments, the combination of SoA particle layout with thread-local grid reduction provided the best overall parallel scalability, achieving near-linear speedup for both the interpolation and mover phases up to 8 threads.